# PUCIT

Punjab University College of Information Technology

Final Documentation
Version 1.0

# School Management System

(SMS)

Team ID- 17

Session: BS Software Engineering, Fall 2019

Project Advisor: Umair Baber


**Submitted By**

Moeez Hayder
BSEF19A537

Atif Aqeel
BSEF19A518

Muhammad Ahsan
BSEF19M508

Shahwar Munir
BSEF19M507

Punjab University College of Information Technology
University of the Punjab, Lahore.

# Statement of Submission

This is to certify that Moeez Hayder (Roll No. BSEF19A537); Atif Aqeel (Roll No. BSEF19A518); Muhammad Ahsan (Roll No. BSEF19M508); Shahwar Munir (Roll No. BSEF19M507) have successfully completed the final project named *School Management System* at the Punjab University College of Information Technology, University of the Punjab, Lahore, in partial fulfilment of the requirements for the degree of Bachelor of Science in Software Engineering.

_____
Project Office Supervisor
PUCIT, Lahore

_____
Project Examiner
Name: _____
Designation: _____
PUCIT

_____
Project Primary Advisor
Name: Umair Baber
Designation: _____
PUCIT

# Proofreading Certificate

It is to certify that I have read the document carefully. I found no spelling, punctuation or grammar mistakes in it. The document is well organised, and I am satisfied that it meets its objectives.

_____
Name: _____
Business Communication and Technical Writing
Lecturer, PUCIT

# Acknowledgement

We thank our project advisor, Umair Baber, for the guidance and feedback that shaped this project from the first proposal to the final report. We also thank the teachers of the Department of Software Engineering at the Punjab University College of Information Technology, whose courses on requirements, design, architecture and quality assurance gave us the methods we used here.

We are grateful to our friends and families for their patience and support while we worked on this project.

Date: _____

# Abstract

Many schools in Pakistan still keep student, teacher and attendance records on paper or in separate spreadsheets. Finding one record takes time, the same data is copied into several files, and a student or teacher must visit the school office to check even a simple fact.

This project builds the School Management System (SMS), a web application that keeps these records in one database and lets each user work with them from a browser. A school registers once and receives an administrator account. The administrator creates classes and subjects, adds students and teachers, and assigns teachers to classes and to the subjects they teach. Every new user receives an email with a link to set a personal password. Teachers mark attendance for a class on a chosen date, and students and staff can check the saved records.

The system uses the MERN stack. The interface is built with React, the server with Node.js and Express, and the data is stored in MongoDB. The server authenticates each request with a JSON Web Token and stores passwords as bcrypt hashes. The menu a user sees depends on the user's role: administrator, teacher or student.

This document describes the goals of the project, its main components, its architecture and its schedule. Fee submission, an announcement panel, parent access and a feedback service are described as optional extensions.

# Table of Contents

Statement of Submission ..... i
Proofreading Certificate ..... ii
Acknowledgement ..... iii
Abstract ..... iv
Table of Contents ..... v
List of Figures ..... vii
List of Tables ..... viii
**Chapter 1: Project Title ..... 1**
1 Project Title ..... 2
**Chapter 2: Project Overview Statement ..... 3**
2 Project Overview Statement ..... 4
2.1 Background ..... 4
2.2 Problem Statement ..... 4
2.3 Project Goal and Objectives ..... 4
2.4 Scope of the Project ..... 4
2.5 Users of the System ..... 5
2.6 Project Team ..... 5
2.7 Project Overview Statement ..... 5
2.8 Project Success Criteria ..... 5
2.9 Assumptions, Risks and Obstacles ..... 5
**Chapter 3: High Level System Components ..... 7**
3 High Level System Components ..... 8
3.1 School Registration and Login ..... 8
3.2 Account Set-Up by Email ..... 8
3.3 Class Management ..... 9
3.4 Subject Management ..... 9
3.5 Student Management ..... 10
3.6 Teacher Management and Assignment ..... 10
3.7 Attendance ..... 11
3.8 Dashboard and Role-Based Navigation ..... 11
**Chapter 4: List of Optional Functional Units ..... 12**
4 List of Optional Functional Units ..... 13
**Chapter 5: Application Architecture ..... 14**
5 Application Architecture ..... 15
5.1 Overview ..... 15
5.2 Client Application ..... 17
5.3 Server Application ..... 17
5.4 Database Design ..... 18

5.5 Main Interaction Flows ..... 21
5.6 Security ..... 24
5.7 Planned Improvements ..... 24
5.8 Deployment and Build Process ..... 24
**Chapter 6: Gantt Chart ..... 25**
6 Gantt Chart ..... 26
**References ..... 28**

# List of Figures

Figure 5.1: System architecture of SMS ..... 16
Figure 5.2: Entity relationship diagram ..... 20
Figure 5.3: Account set-up by email ..... 22
Figure 5.4: Marking attendance ..... 23
Figure 6.1: Gantt chart of the planned schedule ..... 27

# List of Tables

Table 1.1: Basic project details ..... 2
Table 2.1: User roles ..... 5
Table 2.2: Project members ..... 5
Table 2.3: Project overview statement ..... 5
Table 2.4: Risks and how the project reduces them ..... 6
Table 3.1: Menu items by role (from the dashboard sidebar) ..... 8
Table 4.1: Optional functional units ..... 13
Table 5.1: Technology stack ..... 15
Table 5.2: API calls by group ..... 18
Table 5.3: Collections and fields ..... 19
Table 6.1: Planned tasks ..... 26

# Chapter 1
# Project Title

# 1 Project Title

The title of the project is **School Management System**, short name **SMS**.

SMS is a web application for schools. It keeps the records of classes, subjects, students, teachers and daily attendance in one database, and it gives administrators, teachers and students a browser interface for the tasks that belong to their role.

| Item | Detail |
|---|---|
| Project title | School Management System (SMS) |
| Project type | Development |
| Platform | Web application (separate client and server) |
| Technology | MERN stack: MongoDB, Express.js, React, Node.js |
| Degree program | BS Software Engineering, session Fall 2019 |
| Project advisor | Umair Baber |
| Source code | https://github.com/Atif-Aqeel/Final-Year-Project |

Table 1.1: Basic project details

# Chapter 2
# Project Overview Statement

# 2 Project Overview Statement

## 2.1 Background

A school handles a large amount of information every day. It has to know which students are in which class, which teacher teaches which subject, and who was present or absent on a given day. Many schools in Pakistan still record this on paper registers or in separate spreadsheet files kept by different people.

## 2.2 Problem Statement

Paper and spreadsheet records cause five problems that the project addresses:

- Finding the record of one student takes a long time, and paper records can be lost or damaged.
- The same data is written in several places, so copies drift apart and mistakes go unnoticed.
- Nobody can see the attendance of a class or a student over time without counting by hand.
- A student or teacher cannot check personal information without visiting the school office.
- Administrators spend much of their time on routine data entry.

## 2.3 Project Goal and Objectives

The goal of the project is to build an efficient and easy-to-use system for school management in Pakistan. The objectives are:

1. Let the school administration manage student records efficiently.
2. Let students access their own records.
3. Let students and staff view attendance.
4. Create user accounts by email, so that the administrator never needs to know a user's password.
5. Make searching faster and more accurate than manual searching.
6. Reduce the time and cost the administration spends on student affairs.

## 2.4 Scope of the Project

***In scope***

- School registration, login and password set-up by email.
- Management of classes, subjects, students and teachers.
- Assignment of students to classes and of teachers to classes and subjects.
- Daily attendance marking and attendance checking by class and date.
- A dashboard whose menu depends on the user's role.

***Out of scope for the first version***

- Fee submission, grades and results, and parent access.
- An announcement panel and a feedback service.

- Mobile applications. The web interface is the only client.

Chapter 4 lists the out-of-scope items as optional functional units.

### 2.5 Users of the System

| Role | Description |
|---|---|
| Administrator | Registers the school and manages classes, subjects, students and teachers. Can also mark and check attendance. |
| Teacher | Views classes, subjects, students and teachers, and marks and checks attendance. |
| Student | Views classes, subjects and attendance. |

Table 2.1: User roles

### 2.6 Project Team

| Name | Registration # | Email Address |
|---|---|---|
| Moeez Hayder | BSEF19A537 | BSEF19A537@pucit.edu.pk |
| Atif Aqeel | BSEF19A518 | BSEF19A518@pucit.edu.pk |
| Muhammad Ahsan | BSEF19M508 | BSEF19M508@pucit.edu.pk |
| Shahwar Munir | BSEF19M507 | BSEF19M507@pucit.edu.pk |

Table 2.2: Project members

The group leader is Moeez Hayder. The suggested supervisor is Umair Baber.

### 2.7 Project Overview Statement

| Item | Description |
|---|---|
| Project title | School Management System |
| Project goal | Build an efficient system for school management in Pakistan. |
| Type of project | Development |
| Target end users | School administrators, teachers and students |
| Development technology | Object-oriented |
| Platform | Web-based, distributed |
| Suggested supervisor | Umair Baber |

Table 2.3: Project overview statement

### 2.8 Project Success Criteria

The project succeeds if the school system can be used remotely. In practice this means that an administrator can register a school, create its classes, subjects, students and teachers, and that teachers can mark attendance, all from a web browser without installing software.

### 2.9 Assumptions, Risks and Obstacles

| Risk or obstacle | How it is reduced |
|---|---|
| Users may not know how to use the system. | Keep the interface simple, label every form field, and show clear error messages. |
| User data may be exposed to attackers. | Hash passwords, authenticate every request with a token, validate input, and keep secrets in configuration files outside the code. |
| Email or hosting services may fail or change. | Keep these services in small helper modules so that a provider can be replaced. |
| Schools may have poor internet access. | Keep pages light and show a message when a request fails, so that the user can try again. |
| Copyright problems. | Use open-source libraries with permissive licences and original code and text. |
| The scope may grow beyond the available time. | Build the core scope first and treat the optional units as extra work. |

Table 2.4: Risks and how the project reduces them

The project assumes that every school has at least one person who can act as administrator, that users have an email address, and that users have internet access.

# Chapter 3
# High Level System Components

# 3 High Level System Components

The system has eight components. The names of the web pages and API calls in this chapter match the source code.

| Menu group | Item | Admin | Teacher | Student |
|---|---|---|---|---|
| Class | Add Class | Yes | No | No |
| Class | Classes, Class Information | Yes | Yes | Yes |
| Subject | Add Subject, Assign Subject | Yes | No | No |
| Subject | Subjects | Yes | Yes | Yes |
| Student | Add Student | Yes | No | No |
| Student | Students | Yes | Yes | No |
| Teacher | Add Teacher, Assign Teacher | Yes | No | No |
| Teacher | Teachers | Yes | Yes | No |
| Attendance | Mark Attendance | Yes | Yes | No |
| Attendance | Check Attendance | Yes | Yes | Yes |

Table 3.1: Menu items by role (from the dashboard sidebar)

## 3.1 School Registration and Login

### *Purpose*

A school joins the system by registering. After that, every user signs in with an email address and a password.

### *How it works*

- **Sign-up.** The person registering enters the institute name, first name, last name, email and password. The server checks that the email is valid and not used before, that the names are not empty, and that the password has at least 6 characters.
- **Account creation.** The server hashes the password with bcrypt, creates an *Institute* record, and creates a *User* record with the role Admin that belongs to that institute.
- **Token.** The server signs a JSON Web Token that holds the user ID, the institute ID and the role. The browser stores the token and sends it in the *Authorization* header of later requests.
- **Login.** The server looks up the email, compares the password with the stored hash, and returns a new token, the role and the user's name. A wrong password gives the message "Email or Password does not match."
- **Protected pages.** The dashboard checks for a saved token and sends the user to the login page when there is none.

### *Pages and API calls*

Pages: Signup, Login. API calls: *POST /signup*, *POST /login*.

## 3.2 Account Set-Up by Email

***Purpose***

An administrator creates accounts for students and teachers. The user then chooses a personal password, so the administrator never handles it.

***How it works***

1. The administrator adds a student or teacher (see Sections 3.5 and 3.6). The server saves the user with a temporary placeholder password.
2. The server creates a random 32-byte token, hashes it with bcrypt, and builds a link of the form */set-password?token=<hash>&id=<user id>*.
3. The server sends an email from a Handlebars template through Nodemailer and an SMTP account. The message says that a new account was created and contains a Set Password link.
4. The server stores the plain token on the user record.
5. The user opens the link and enters a new password of at least 5 characters. The server compares the token from the link with the stored one using bcrypt, saves the new password hash, clears the token, and signs the user in.

A second template for a password reset email exists in the code base for later use.

***Pages and API calls***

Page: Set Password. API call: *POST /set-password*. Figure 5.3 shows the full sequence.

### 3.3 Class Management

***Purpose***

A class (for example, "Grade 5") groups students and the teachers who teach them.

***How it works***

- **Add class.** The administrator enters a class name. The server rejects a name that already exists in the same school.
- **List classes.** The Classes page shows every class with the number of students and the number of teachers in it.
- **Class information.** The Class Information page shows one class with its students (name, roll number, admission date) and its teachers (name, staff ID, joining date).

***Pages and API calls***

Pages: Add Class, Classes, Class Information. API calls: *POST /addClass*, *GET /getClasses*, *GET /getClassesData*, *GET /getClassData*.

### 3.4 Subject Management

***Purpose***

Subjects are the courses a school teaches. The system records which teacher teaches which subject in which class.

***How it works***

- **Add subject.** The administrator enters a subject name. The subject belongs to the administrator's school.

- **List subjects.** The Subjects page shows each subject with the number of classes and teachers linked to it.
- **Subject details.** A subject page lists the classes and teachers linked to it.
- **Assign subjects.** The administrator picks a class, a teacher and one or more subjects. The server deletes the teacher's earlier subjects in that class and saves the new selection as *SubjectTeacherClass* records.

***Pages and API calls***

Pages: Add Subject, Subjects, Subject Details, Assign Subject. API calls: *POST /subject*, *GET /subject*, *GET /subject/all*, *GET /subject/classTeacherSubjects*, *POST /subject/assign*.

### 3.5 Student Management

***Purpose***

This component holds the records of students and their class.

***How it works***

- **Add student.** The administrator enters email, first name, last name, date of birth, gender, admission date, roll number and class. The server checks that the email is new, that the class exists, and that the roll number is not taken in that class. It then saves the student, adds the student to the class roster, and starts the account set-up email (Section 3.2).
- **List students.** The Students page lists all students of the school with roll number, admission date and class, sorted by class name.
- **Student profile.** A profile page shows the stored details of one student. Email and password are not returned.
- **Change class.** The administrator can move a student to another class. The server removes the student from the old class roster and adds the student to the new one.

***Pages and API calls***

Pages: Add Student, Students, student profile. API calls: *POST /addStudent*, *GET /getStudents*, *GET /getUser*, *POST /updateStudentClass*.

### 3.6 Teacher Management and Assignment

***Purpose***

This component holds the records of teachers and the classes they teach.

***How it works***

- **Add teacher.** The administrator enters email, staff ID, role, first name, last name, date of birth, gender and joining date. The server checks that the email is new and that the staff ID is not used in the school, then starts the account set-up email.
- **List teachers.** The Teachers page shows each teacher with staff ID, joining date and the number of classes.
- **Assign classes.** On the Assign Teacher page the administrator selects the classes of a teacher. The server checks that every class exists, saves the list on the teacher, and updates the teacher lists of the classes that were added or removed.

***Pages and API calls***

Pages: Add Teacher, Teachers, Assign Teacher. API calls: *POST /addTeacher*, *GET /getTeachers*, *GET /getTeachersWithClasses*, *GET /getClassTeachers*, *POST /updateTeacherClasses*.

### 3.7 Attendance

***Purpose***

Teachers record who is present in a class on a given day. Anyone with access can look the records up later.

***How it works***

- **Mark attendance.** The teacher picks a class and a date and presses "Get Class Data". The page lists the students of the class. The teacher marks each student present or absent, or sets the whole class to present or absent with one button, and then saves.
- **Check attendance.** The Check Attendance page uses the same screen in read-only mode and shows the saved records for the chosen class and date.
- **Stored data.** Each record in the *Attendance* collection holds the date, the student, the class and a present flag.
- **Validation.** The date must be a valid date and the class must exist.

***Pages and API calls***

Pages: Mark Attendance, Check Attendance. API calls: *GET /getInitialAttendance*, *POST /markAttendance*, *GET /getAttendance*. Figure 5.4 shows the sequence.

### 3.8 Dashboard and Role-Based Navigation

***Purpose***

The dashboard is the main screen after login. It gives each user quick access to the pages that belong to the user's role.

***How it works***

- A sidebar groups the pages under Class, Subject, Student, Teacher and Attendance.
- The browser stores the user's role at login. The sidebar hides every item that the role may not use, and hides a whole group when none of its items apply (Table 3.1).
- A shared header, form components, loading indicator and table component keep the pages consistent.

# Chapter 4
# List of Optional Functional Units

## 4 List of Optional Functional Units

The following units are not part of the first version. They will be built only if time allows.

| Unit | Description |
|---|---|
| Fee submission | Record fee payments for each student and let students and administrators view the payment history. |
| Grades and results | Let teachers enter marks for a subject and let students see their results. |
| Parent access | Let a parent follow the attendance and results of a child during the year. |
| Announcement panel | Show announcements about school events and extracurricular activities. |
| Feedback service | Let users send feedback about the management of the school. |

Table 4.1: Optional functional units

# Chapter 5
# Application Architecture

# 5 Application Architecture

## 5.1 Overview

SMS is a three-tier client-server application. A React single-page application runs in the user's browser. It sends requests to a REST API built with Express on Node.js. The API reads and writes data in a MongoDB database and sends emails through an SMTP service. Figure 5.1 shows the tiers and the data that moves between them.

| Layer | Technology | Use in the project |
|---|---|---|
| Client | React 17, React Router 6 | Pages, routes and components |
| Client | Formik, Axios | Forms with validation; HTTP calls to the API |
| Client | Sass, Tailwind CSS | Styles for buttons, forms, tables and the sidebar |
| Server | Node.js, Express | REST API and routing |
| Server | express-validator | Checks every request before it reaches a controller |
| Server | jsonwebtoken, bcrypt | Tokens for sign-in; password hashing |
| Server | Nodemailer, Handlebars | Password set-up emails built from templates |
| Database | MongoDB, Mongoose | Document storage, schemas and references |
| Tooling | Git, GitHub, GitHub Actions | Version control and an automatic build and test run |
| Hosting | Vercel | Deployment of the Node.js server |

Table 5.1: Technology stack

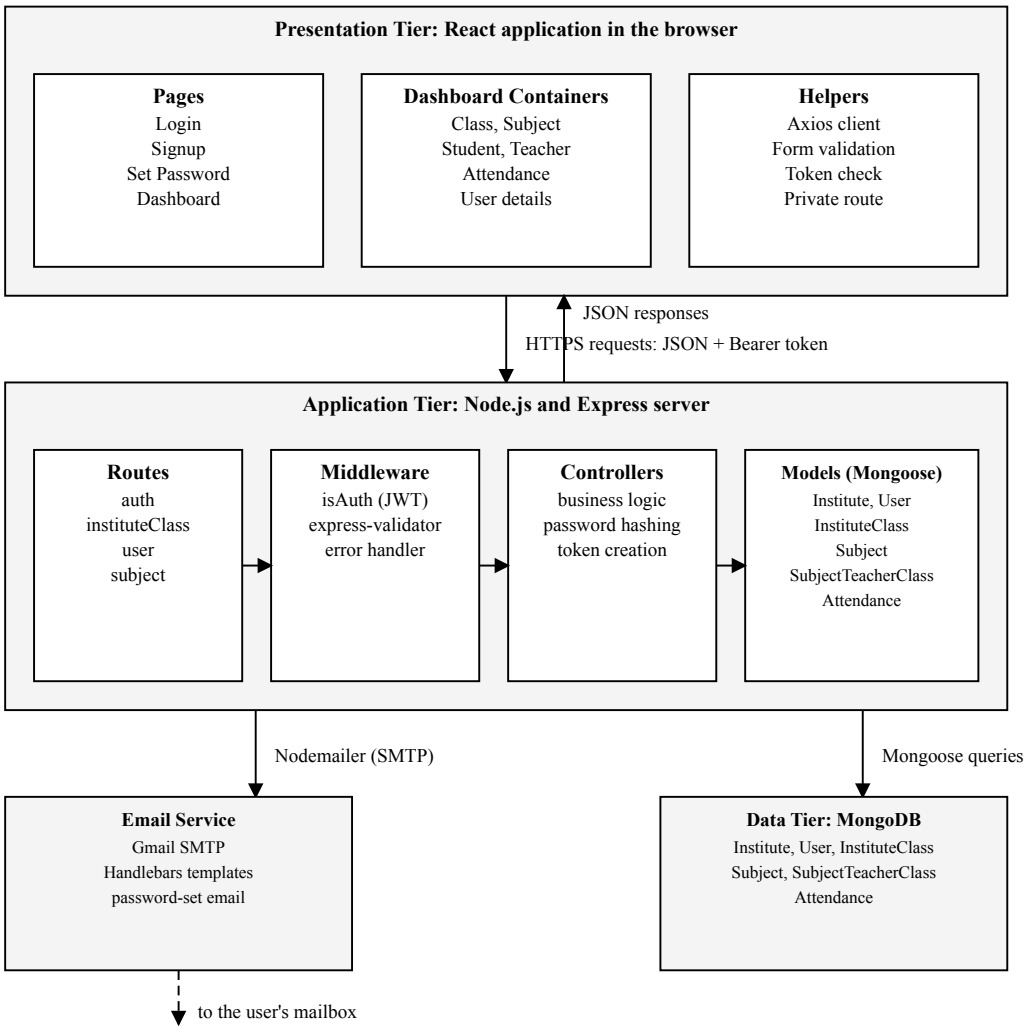



Figure 5.1: System architecture of SMS

### 5.2 Client Application

The client was created with Create React App. The source folder is split into *pages* (Login, Signup, Set Password, Dashboard), *containers* (one folder each for class, subject, student, teacher, attendance and user pages), *components* (sidebar, header, form fields, loading indicator) and *helpers* (the Axios instance, form validation, the token check). React Router maps each address to a page. The dashboard routes are nested under */dashboard*.

Forms use Formik. Each field is validated in the browser first. When the server sends back validation errors, the form shows them next to the matching field.

### 5.3 Server Application

The server is an Express application with four route groups: authentication, classes, users and subjects. A request to a protected route passes through three steps:

1. The *isAuth* middleware reads the Bearer token from the *Authorization* header, verifies it, and puts the user ID, institute ID and role on the request. A missing or invalid token gives status 401.
2. express-validator rules check the input and query the database where needed, for example to confirm that a class exists or that an email is not taken.
3. The controller runs the business logic and returns a JSON response.

A shared error handler turns thrown errors into JSON with a status code and a message. Validation errors include the failing field names, which the client uses to mark the fields.

| Group | Method and path | Purpose |
|---|---|---|
| Authentication | POST /signup | Register a school and its first administrator |
| Authentication | POST /login | Sign in |
| Authentication | POST /set-password | Set a password from the email link |
| Classes | POST /addClass | Create a class |
| Classes | GET /getClasses | List class names |
| Classes | GET /getClassesData | List classes with student and teacher counts |
| Classes | GET /getClassData | Get one class with its students and teachers |
| Users | POST /addStudent | Create a student |
| Users | POST /addTeacher | Create a teacher |
| Users | GET /getUser | Get one user profile |
| Users | GET /getStudents | List students |
| Users | GET /getTeachers | List teachers |
| Users | GET /getTeachersWithClasses | List teachers with their classes |
| Users | GET /getClassTeachers | List the teachers of a class |
| Users | POST /updateStudentClass | Move a student to another class |
| Users | POST /updateTeacherClasses | Set the classes of a teacher |
| Users | GET /getInitialAttendance | Get the student list for an attendance date |
| Users | POST /markAttendance | Save attendance for a class and date |
| Users | GET /getAttendance | Get saved attendance |
| Subjects | POST /subject | Create a subject |
| Subjects | GET /subject | Get one subject with its classes and teachers |
| Subjects | GET /subject/all | List subjects |
| Subjects | GET /subject/classTeacherSubjects | Get the subjects a teacher has in a class |
| Subjects | POST /subject/assign | Assign subjects to a teacher in a class |

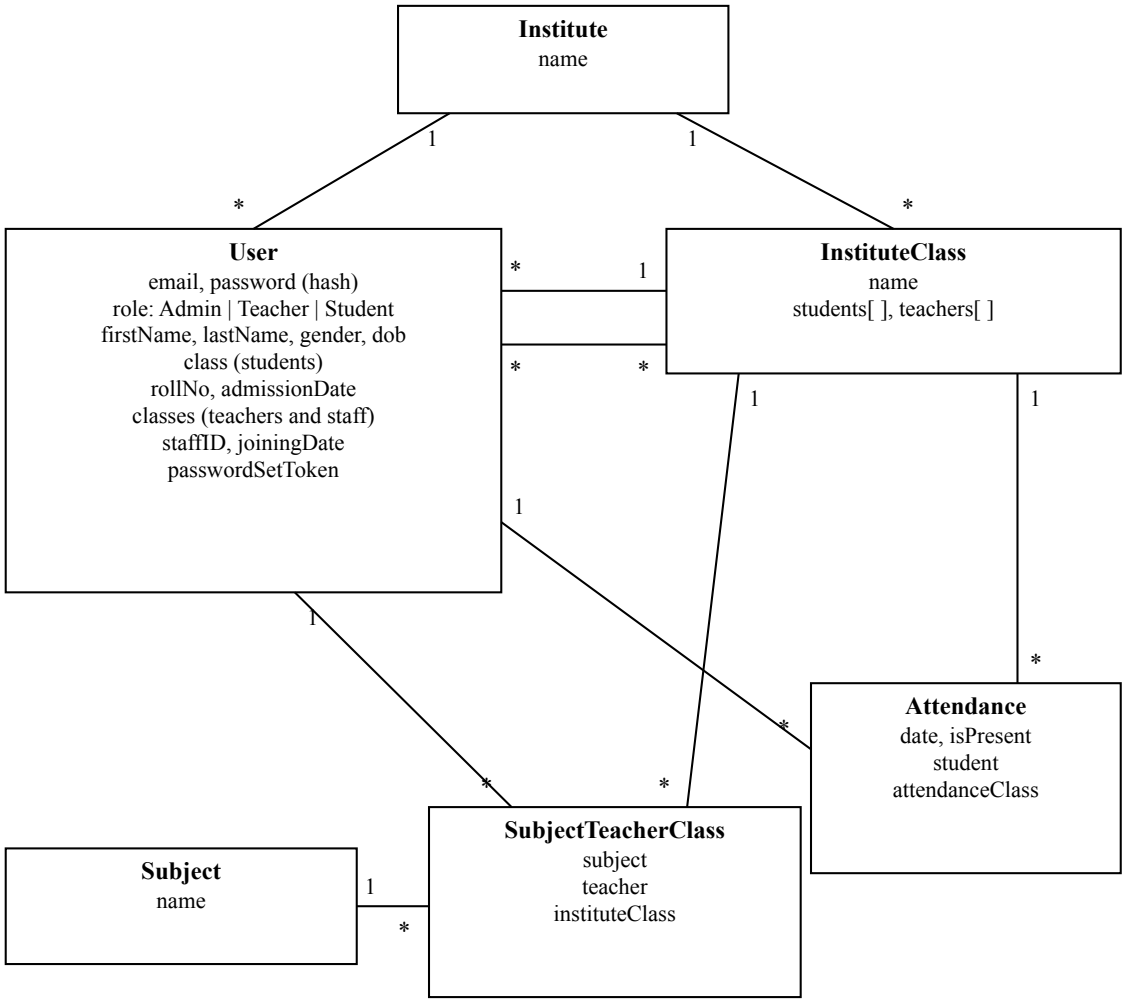
Table 5.2: API calls by group

### 5.4 Database Design

MongoDB stores the data in six collections. Mongoose schemas define the fields, and references between documents use object IDs. Figure 5.2 shows the collections and how they relate.

| Collection | Main fields | Relationships |
|---|---|---|
| Institute | name | Has many students, admins, teachers and classes |
| User | email, password (hash), role (Student, Teacher or Admin), firstName, lastName, gender, dob; student fields: class, rollNo, admissionDate; staff fields: classes, staffID, joiningDate; passwordSetToken | Belongs to one institute. A student belongs to one class. A teacher can teach in many classes. |
| InstituteClass | name | Belongs to one institute. Has many students and teachers. |
| Subject | name | Belongs to one institute. |
| SubjectTeacherClass | subject, teacher, instituteClass, institute | Links one subject, one teacher and one class. |
| Attendance | date, isPresent, student, attendanceClass | One record per student, class and date. |

Table 5.3: Collections and fields




1 = one, * = many. Every record that belongs to a school also stores that school (institute).
User.class links a student to one class (many to one).
User.classes links a teacher to many classes, and each class lists its teachers (many to many).
SubjectTeacherClass links one subject, one teacher and one class.

Figure 5.2: Entity relationship diagram

### 5.5 Main Interaction Flows

Two flows show how the tiers work together. Figure 5.3 shows how a new user gets an account and signs in for the first time. Figure 5.4 shows how a teacher marks attendance.

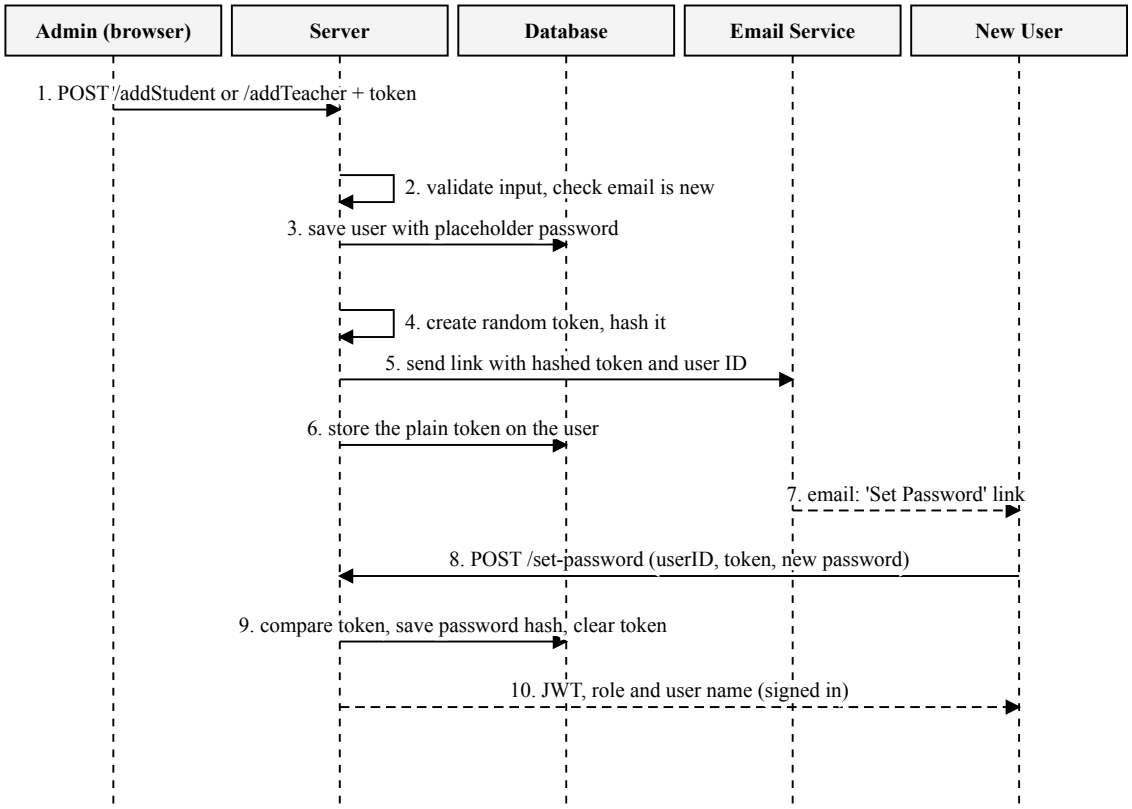



Figure 5.3: Account set-up by email

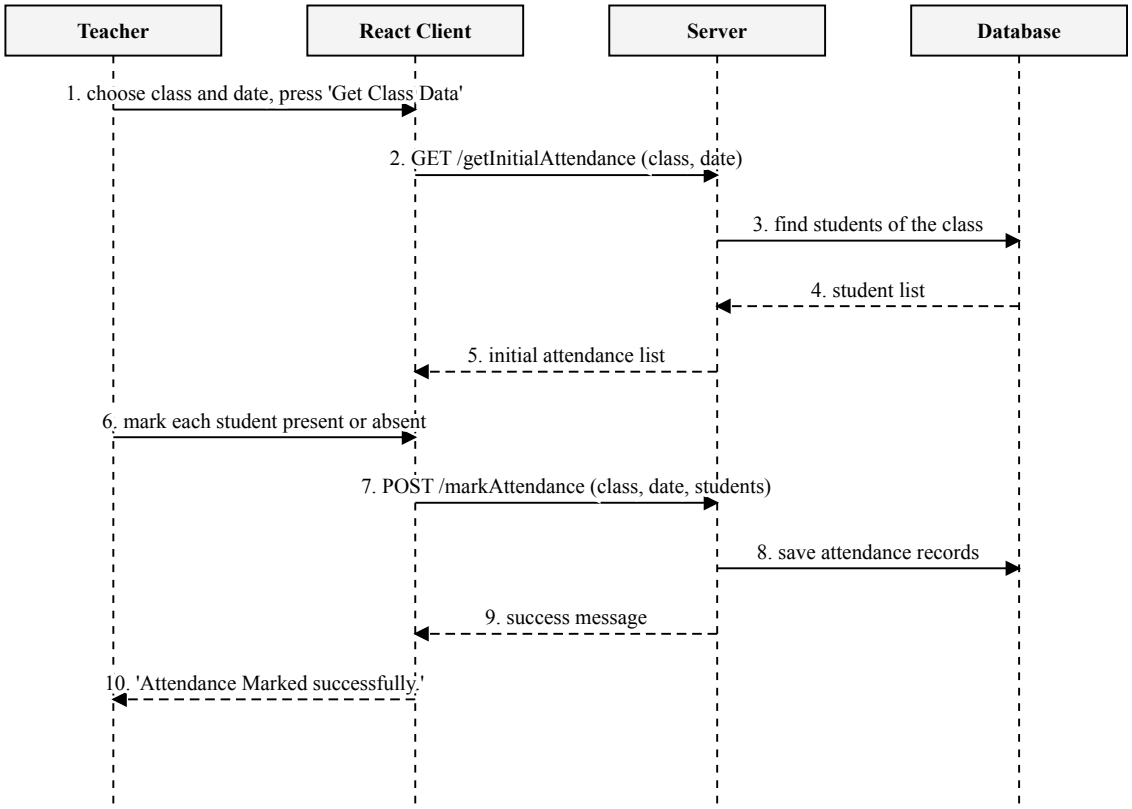



Figure 5.4: Marking attendance

### 5.6 Security

- Passwords are stored only as bcrypt hashes.
- A signed JSON Web Token authenticates every request to a protected route.
- The token carries the institute ID, and the controllers use it to select the records of the user's own school.
- express-validator checks the input of every write request.
- Secrets such as the database address, the token secret and the email password are read from configuration files and are not written in the code.

### 5.7 Planned Improvements

Two changes are planned for a later version. First, the server should check the user's role on each route. At present the role decides which menu items the interface shows, while the server checks that the token is valid. Second, the server should limit cross-origin access to the address of the client application instead of allowing every origin.

### 5.8 Deployment and Build Process

The server contains a *vercel.json* file for deployment on Vercel. A GitHub Actions workflow runs on every push and pull request to the main branch. It installs the dependencies, builds the project and runs the tests on Node.js 14, 16 and 18.

# Chapter 6
# Gantt Chart

# 6 Gantt Chart

The project runs over two semesters. Final Year Project I (FYP-I) covers weeks 1 to 16, and Final Year Project II (FYP-II) covers weeks 17 to 32. Table 6.1 lists the tasks and Figure 6.1 shows them on a time line.

| Task | Start week | End week |
|---|---|---|
| Proposal and requirements | 1 | 8 |
| System design | 5 | 16 |
| Backend: authentication, classes, subjects, users | 9 | 28 |
| Frontend: login, dashboard, management pages | 9 | 28 |
| Attendance module and email set-up flow | 21 | 28 |
| Testing and bug fixing | 21 | 32 |
| Deployment, documentation, final report | 29 | 32 |

Table 6.1: Planned tasks

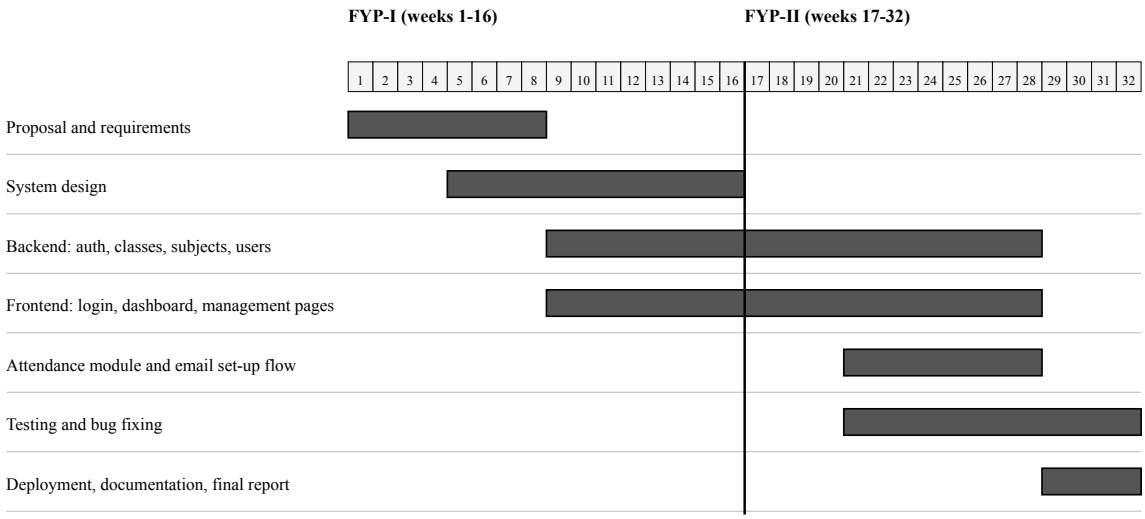



Figure 6.1: Gantt chart of the planned schedule

# References

[1] Jones, M., J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, May 2015. https://www.rfc-editor.org/rfc/rfc7519

[2] Provos, N., and D. Mazieres. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference, 1999, pp 81-91.

[3] React documentation. Meta Open Source. https://react.dev/

[4] Express documentation. OpenJS Foundation. https://expressjs.com/

[5] MongoDB documentation. MongoDB, Inc. https://www.mongodb.com/docs/

[6] Mongoose documentation. https://mongoosejs.com/docs/

[7] Nodemailer documentation. https://nodemailer.com/

[8] School Management System source code. https://github.com/Atif-Aqeel/Final-Year-Project